

UpDLRM: Accelerating Personalized Recommendation using Real-World PIM Architecture

Sitian Chen¹, Haobin Tan², Amelie Chi Zhou^{1*}, Yusen Li³, Pavan Balaji⁴

¹Hong Kong Baptist University ²Shenzhen University ³Nankai University ⁴Meta

ABSTRACT

Deep Learning Recommendation Models (DLRMs) have gained popularity in recommendation systems due to their effectiveness in handling large-scale recommendation tasks. The embedding layers of DLRMs have become the performance bottleneck due to their intensive needs on memory capacity and memory bandwidth. In this paper, we propose *UpDLRM*, which utilizes real-world processing-in-memory (PIM) hardware, UPMEM DPU, to boost the memory bandwidth and reduce recommendation latency. The parallel nature of the DPU memory can provide high aggregated bandwidth for the large number of irregular memory accesses in embedding lookups, thus offering great potential to reduce the inference latency. To fully utilize the DPU memory bandwidth, we further studied the embedding table partitioning problem to achieve good workload-balance and efficient data caching. Evaluations using real-world datasets show that, UpDLRM achieves much lower inference time for DLRM compared to both CPU-only and CPU-GPU hybrid counterparts.

1 INTRODUCTION

Deep Learning Recommendation Models (DLRMs) have become widely utilized for click-through rate (CTR) prediction and news rankings [5, 13]. According to Meta, DLRM models account for more than 60% of their AI inference cycles in production, making them a primary target for optimization [7]. However, performance optimization for DLRMs is challenging, mainly due to their intensive needs on memory capacity and bandwidth [19]: 1) The number of parameters in DLRMs is much larger than traditional deep learning models due to the *embedding layers* which map high-dimensional categorical features (e.g., user IDs) to lower-dimensional dense vectors. The categorical features are encoded using embedding tables which can be very large in real-world scenarios (e.g., 10s of GBs). 2) During inference, DLRMs perform multiple *embedding lookups* and *reductions*, requiring frequent and irregular memory accesses. These contribute significantly to the memory footprint and introduce memory bottlenecks, leading to increased latency.

Due to the limited memory capacity of GPUs, existing studies mainly adopt CPU-GPU hybrid architecture for DLRM inference, where the embedding tables (EMTs) are stored in CPU memories and GPUs are employed for highly parallel computations [13, 20].

Some recent work proposed to load popular data into limited GPU memory to further reduced data access latency [4]. However, the hybrid architecture introduces unavoidable communication between CPUs and GPUs, and may cause GPU stalls due to the memory bottleneck of CPUs. Since DLRM inference is less compute-intensive compared to training, recent studies [8] found it is possible to make real-time recommendations at scale based on only CPUs. However, memory is still a major performance bottleneck and workload-aware pre-processing (e.g., prefetching, caching) are needed to reduce inference latency [8]. In this paper, we proposed to boost the memory bandwidth of CPUs using real-world processing-in-memory (PIM) hardware, UPMEM DPU [2].

UPMEM DPU is equipped with dedicated processing units connected closely to memory cells (details in Section 2.2). We propose to store the large EMTs using DPU memory and perform costly memory lookups and reductions using DPUs. This design offers two compelling benefits. *First*, by offloading the memory-bound embedding operations to DPUs, we can reduce the resource contention on CPU memory bandwidth and hence accelerate the inference time. *Second*, the parallel nature of DPU memory banks allows for efficient processing of multiple embedding lookups and reductions simultaneously, further accelerating the inference time.

There exist several design issues for efficiently utilizing DPUs. Since the capacity of each DPU memory bank (MRAM) is limited to 64MB, it is usually necessary to partition an EMT onto multiple DPUs. When partitioning EMTs, we have to jointly consider the architectural features of DPUs and the memory access patterns of embedding operations to efficiently utilize the DPU memory bandwidth. *First*, UPMEM DIMM does not support inter-DPU communication directly. Data movement across DPUs has to go through CPU memory. Thus, when partitioning EMTs, we should avoid frequent inter-DPU data exchange. *Second*, each DPU core supports multi-threads and pipeline execution. We should fully utilize this hardware feature to improve the efficiency of DPU-supported embedding operations. *Third*, the popularities of items differ in real-world datasets [20]. Thus, when items in EMTs are partitioned onto different DPUs, we have to consider the aggregated access frequency of each partition to achieve workload balance.

In this paper, we propose *UpDLRM*, which utilizes UPMEM PIM architecture to accelerate the memory-intensive embedding layers of DLRM, in order to achieve low inference time. Considering the above mentioned design issues, we study the EMT partitioning problem at three different levels. First, we consider the EMT partitioning only consider the hardware features, in order to fully utilize the memory bandwidth and maximize the throughput for embedding lookup operations (Section 3.1). Second, we take the imbalanced data access frequencies into account, and proposed non-uniform EMT partitioning to achieve workload balance among DPUs. This ensures efficient utilization of DPU resources and minimizes embedding layer processing time (Section 3.2). Third, we further consider

*Corresponding author: Amelie Chi Zhou.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

DAC '24, June 23–27, 2024, San Francisco, CA, USA

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0601-1/24/06.

<https://doi.org/10.1145/3649329.3658266>

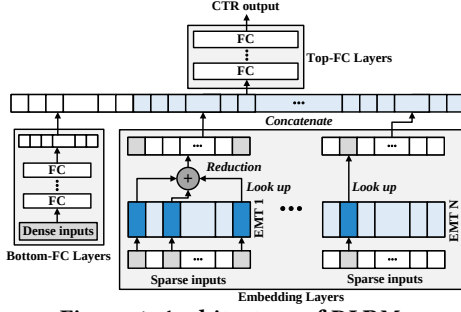


Figure 1: Architecture of DLRMs

the co-occurrence feature of multiple items in real recommendation systems, and adopt existing caching technique [20] to further reduce memory traffic. To deal with the workload imbalance problem introduced by data caching, we proposed a cache-aware partitioning method that jointly balance the memory accesses on cache storage and regular EMT storage (Section 3.3). Evaluations on six real-world datasets and four DLRM implementations using different hardware architectures have demonstrated the effectiveness of UpDLRM on optimizing the inference time. Overall, it achieved up to 4.6x speedup on inference time compared to the other comparisons.

2 BACKGROUND AND MOTIVATION

2.1 DLRM Model

Figure 1 shows the structure of DLRMs. The bottom-FC layers process “dense” inputs that are usually continuous attributes (e.g., age, price, rating) to generate dense features. The embedding layers handle “sparse” inputs that are usually categorical features (e.g., user ID, gender, location) encoded using one-hot or multi-hot to generate embeddings. Each embedding layer contains a large embedding table (EMT) responsible for storing feature vectors of a specific category. After obtaining the dense and sparse features, they are concatenated and fed into the top FC layer to predict the CTR. The embedding layer is the main component that causes high memory capacity and bandwidth requirements of DLRMs.

Memory Capacity Requirement: The EMTs are essentially lookup tables, where each row in the table represents the embedding vector associated with a specific value of the category. Hence the size of a EMT is determined by the number of distinct values in a categorical feature. For example, consider the category being user ID. The number of users in a commercial recommendation system can easily reach billions (e.g., over 3 billions in Meta [14]). Assume the number of users is one billion and the embedding dimension is 20. The size of the table is $10^9 * 20 * 4\text{bytes}$, namely 80GB. Since there are usually many categories considered in real-world recommendation systems, the memory consumption of EMTs is substantial.

Memory Bandwidth Requirement: During DLRM inference, the embedding layer takes the input categorical values (indices) and performs a *lookup operation* in the EMT to retrieve corresponding embedding vectors. This operation requires fetching embedding vectors from different memory locations based on the input indices, which can result in frequent memory accesses and require high bandwidth to sustain the data transfer rates. In real-world scenarios, the popularity of different categorical values can vary significantly [20]. This can lead to imbalanced accesses to different memory locations in the EMTs.

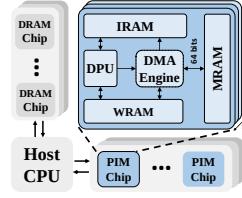


Figure 2: UPMEM's PIM architecture

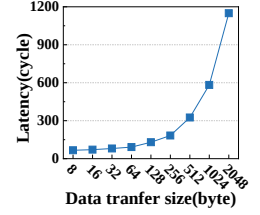


Figure 3: MRAM read latency

2.2 UPMEM PIM Architecture

The UPMEM PIM architecture is the first real-world PIM system commercialized in 2019 [2, 6]. Figure 2 plots the architecture of UPMEM's PIM system. A UPMEM module consists of a standard DDR4 DIMM module that incorporates 8-16 PIM chips. Each chip comprises 8 processing cores known as DRAM Processing Units (DPUs). Each DPU has exclusive access to specific memory components, including a 64-MB DRAM bank MRAM, a 24-KB instruction memory IRAM, and a 64-KB scratchpad memory WRAM. The MRAM can be accessed by the host CPU for transferring input data (main memory to MRAM) and retrieving results (MRAM to main memory). These data transfers can occur concurrently if the buffers transferred to and from all MRAM banks are of the same size. Otherwise, the transfers happen sequentially. It's important to note that inter-DPU communication relies on the host CPU.

Different from RRAM-based PIM architectures [11, 19], which perform arithmetic operations using NOR gates within the RRAM arrays, UPMEM's PIM is the first “real” process in memory hardware that contains dedicated processing units, namely DPUs. Each DPU is a multi-threaded 32-bit RISC core with its own specific ISA [3] to facilitate data transfers between the MRAM and the WRAM. The maximum MRAM-WRAM bandwidth per DPU can achieve approximately 800 MB/s [2]. All DPUs in the UPMEM module function together as a parallel co-processor to the host CPU.

2.3 Motivation

Embedding layers are the primary performance bottlenecks of DLRM inference due to their high requirements on memory capacity and memory bandwidth. The PIM architecture can be utilized to alleviate the memory bandwidth bottleneck and process embedding data near memory. Additionally, the UPMEM technical paper [2] indicates that the total cost of ownership (TCO) gain for the PIM platform is estimated to be around 10x, with a potential reduction of 60% in energy consumption. Thus, studying PIM-accelerated DLRM inference could potentially lead to cost- and energy-efficient solution to industry. In this paper, we propose to store the large embedding tables in the MRAM of DPUs and perform lookup operations using near memory DPU cores. Since each DPU MRAM has a limited capacity of 64MB, an EMT that is larger than 64MB has to be partitioned first to fit in the DPU memory. The partitioning has to consider several design issues as discussed next.

3 DESIGN DETAILS

3.1 Embedding Table Partitioning

We partition each embedding table into tiles smaller than 64MB to fit in the DPU memories. To simplify the problem, let's first consider

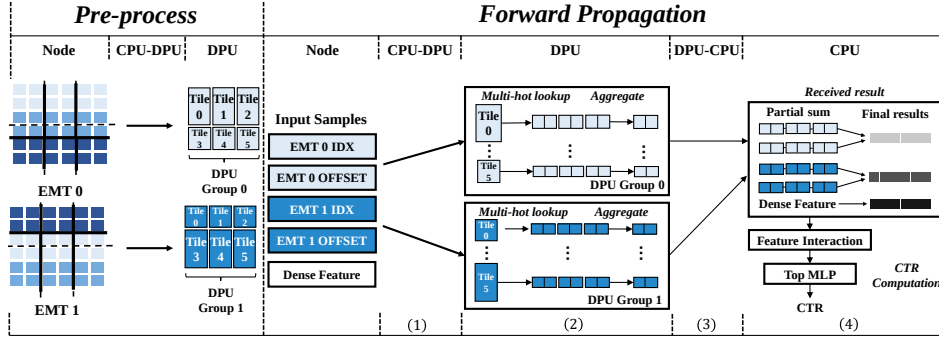


Figure 4: Pre-processing and forward propagation stages of our DPU-based DLRM implementation

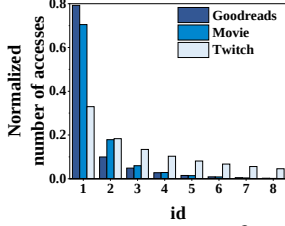


Figure 5: Proportion of partitions being accessed

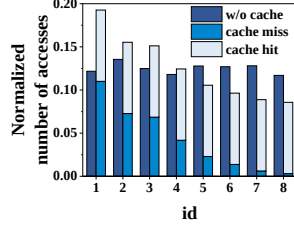


Figure 6: Access pattern w/ and w/o cache using Movie

uniform partitioning, namely all tiles have the same number of rows (denoted as N_r) and columns (denoted as N_c). Assume each feature value is represented using 32-bit data, we have $N_r * N_c * 4B \leq 64MB$.

As shown in Figure 4, during the forward propagation, given a batch of embedding table indices and offsets (multi-hot vectors), the corresponding feature data are fetched from DPUs and transferred back to CPU. The feature data fetching in stage 2 contains two steps. First, the multi-hot lookup operation searches in DPU MRAMs for corresponding feature data specified by IDX and OFFSET. Second, multiple vectors are returned and aggregated within DPU to utilize its computation power. Due to EMT partitioning, the aggregated embedding vectors are only partial results and need to be transferred back to the CPU to generate final feature vectors. We study the time spent on embedding layers as the sum of CPU-DPU communication time (stage 1), embedding lookup time (stage 2) and DPU-CPU communication time (stage 3).

Each embedding lookup operation reads $N_c * 4$ bytes of data from MRAM at the same time. We study the performance of each memory access on MRAM as shown in Figure 3. Note that each MRAM read has to be 8 bytes aligned and can be 2,048 bytes maximum. Interestingly, we found that when the size of data increases from 8B to 32B, the latency increases rather slowly. After 32B, the latency increases more dramatically. The partitioning approach should carefully consider this feature when deciding the tile size. That is, we prefer to have $N_c * 4B \leq 32B$, namely $N_c \leq 8$.

Multiple DPUs perform lookup operations concurrently. Assuming balanced distribution of memory accesses in the dataset, uniform EMT partitioning can result in the same *embedding lookup time* on each DPU. Approximately, we can estimate the lookup time on each DPU as $T_{lkp} = \frac{N_r}{R} * batch_size * Avg_Red * t_a$, where t_a represents the memory access time indicated in Figure 3 and Avg_Red represents the average reduction in the number of active features (ones) in the multi-hot encoding. Similarly, the *communication time*

for transferring indices from CPU to DPU memory can be estimated as $T_{c-comm} = \frac{N_r}{R} * batch_size * Avg_Red * t_c$, where t_c represents the time to transfer one value from CPU to DPU. The time for transferring partial results from DPU to CPU memory can be estimated as $T_{d-comm} = N_c * batch_size * t_d$, where t_d represents the time to transfer one value from DPU to CPU. Thus, to minimize the time on embedding layers, we aim to find a good tradeoff between N_r and N_c to achieve the following goal:

$$\min T_{c-comm} + T_{lkp} + T_{d-comm} \quad (1)$$

$$\text{s.t.,} \quad N_r * N_c = \frac{R * C}{N_{dpu}} \leq 1.6 * 10^7 \quad (2)$$

$$N_c = 2k, \quad 1 \leq k \leq 4 \quad (3)$$

where N_{dpu} represents the number of DPUs used to stored EMTs. R and C represent the number of rows and columns of the EMT, respectively. Both N_{dpu} and $R * C$ are fixed inputs to the problem. The constraints greatly reduce the solution space of the problem. Thus we can simply search for the best N_r and N_c exhaustively.

3.2 Non-Uniform EMT Partitioning

In the above discussion, we assume balanced distribution of memory accesses in a dataset, which is usually not true in real-world applications. We study the data access patterns of EMTs using three real-world datasets Goodreads [17, 18], Movie [15] and Twitch [16]. Figure 5 shows the total number of accesses per row block when dividing the rows into 8 blocks. Clearly, the three datasets all show imbalanced access distributions. The most popular block has 340 times higher number of accesses compared to the least popular block. This will lead to workload imbalance problem to DPUs and make the uniform partitioning no longer optimal.

To accommodate this, we propose a non-uniform EMT partitioning method that considers the imbalanced data access pattern in real datasets. Specifically, consider using the same number of DPUs as the uniform partitioning, our non-uniform partitioning cuts an EMT into tiles of varying sizes and stores each tile in a separate DPU. DPUs used to store the same EMT collectively form a group as shown in Figure 4. The goal is to achieve optimal workload balance among DPUs. We adopt a greedy method to achieve this goal. Specifically, by profiling the historical user-item access trace, we can obtain the access frequencies of all items. Consider each DPU as a bin, the problem of assigning each row to DPUs is a classical Bin-Packing problem with a fixed number of bins. We first order items according to their frequencies and then iteratively

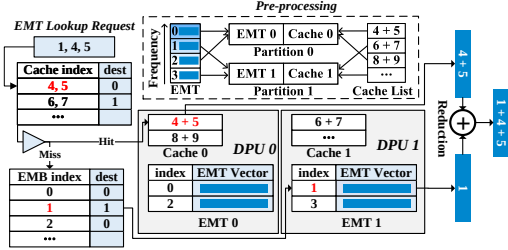
Algorithm 1 Cache-Aware Non-Uniform Partitioning

Input: N_{dpu} : The number of DPUs/partitions;
 obj_freq : The item access frequency trace;
 $cache_res$: A list of cached partial sums generated by GRACE [20].
Output: $assigned_part$: Assigned partition of each item.

```

1: Sort  $obj\_freq$  in descending order;
2:  $p\_id = 0$ ;
3:  $part\_count$  counts the current size of each partition;
4: for  $list$  in  $cache\_res$  do ▷ Cache hit
5:    $benefit = list[-1]$ ; ▷ Reduced memory access estimated by GRACE
6:    $p\_id \leftarrow$  The partition with the lowest  $part\_count$  value and enough cache capacity;
7:   for  $index$  in  $list$  do
8:      $assigned\_part[index] = p\_id$ ;
9:      $part\_count[p\_id] += obj\_freq[index]$ ;
10:   $part\_count[p\_id] -= benefit$ ;
11: for  $index$  in  $obj\_freq$  do ▷ Cache miss
12:  if  $index \notin cache\_res$  then
13:     $p\_id \leftarrow$  The partition with the lowest  $part\_count$  value and enough EMT capacity;
14:     $assigned\_part[index] = p\_id$ ;
15:     $part\_count[p\_id] += obj\_freq[index]$ ;
16: return  $assigned\_part$ ;

```

**Figure 7: Example of cache-aware partitioning**

assign each row to the bin with the lowest total frequency unless the bin is full (64MB capacity). Note that we adopt the same N_c as optimized in uniform partitioning. The complexity of the algorithm is hence $O(R)$. One could batch items when doing the assignment to reduce algorithm complexity. Consider the Movie dataset which has a skewed item access frequency as shown in Figure 5. When partitioning the EMT into 8 row blocks with our non-uniform algorithm, it can be observed from Figure 6 that the number of accesses per partition is much more balanced (i.e., w/o cache).

3.3 Cache-Aware Non-Uniform Partitioning

Existing studies [9, 20] leverage the power-law distribution of item access frequencies to cache the partial sum of popular items to reduce memory traffic and accelerate DLRM inference. For example, the recent GRACE [20] algorithm takes the *co-occurrences* of popular items into account, and adopts a graph-based algorithm to identify frequently accessed item combinations. These combinations' partial sums are cached to reduce memory traffic. The caching techniques have already demonstrated effectiveness on reducing DLRM inference latency. However, when applied to our DPU-based DLRM, they further exacerbate the workload imbalance problem among DPUs. As shown in Figure 6, when partitioning the Movie dataset using our non-uniform algorithm with GRACE-based caching, the total number of memory accesses is reduced by 40% compared to without caching, while the access pattern is much imbalanced across DPUs.

To address this issue, we propose cache-aware non-uniform partitioning as shown in Algorithm 1. The function takes three inputs, including N_{dpu} , obj_freq which records the access frequency history of different items, and $cache_res$ which is a cache list containing partial sums generated by existing caching technique [20].

Table 1: Workload Configurations

Category	Workload	Avg.Reduction	#Items
Low Hot	AmazonClothes(clo) [15]	52.91	2,685,059
	AmazonHome(home) [15]	67.56	1,301,225
Medium Hot	MetaFBGEMM1(meta1) [1]	107.2	5,783,210
	MetaFBGEMM2(meta2) [1]	188.6	5,999,981
High Hot	GoodReads(read) [17, 18]	245.8	2,360,650
	GoodReads2(read2) [17, 18]	374.08	2,360,650

A cache list of $\{a, b, c\}$ means that items a , b and c frequently co-exist in the same sample, so that partial sums a , b , c , $a + b$, $a + c$, $b + c$ and $a + b + c$ are cached to reduce the memory traffic. We divide each MRAM capacity into two parts: one storing embedding vectors and the other storing the cached partial sums. Initially, we sort obj_freq in descending order. That is, items with higher frequencies are ranked in front. To decide the partition of an item, we first look for it in the cache list (Line 4). If it exists in the list, we store the item in a partition with the minimum total frequency and has enough capacity for cache storage (Line 5-10). If the item does not exist in cache list, we store it in the EMT part of MRAM storage (Line 12-15). With this greedy algorithm, we aim to balance the combined number of memory accesses (EMT+cache) per partition.

Figure 7 illustrates an example of our cache-aware partitioning algorithm. Consider a sample request to look up indices 1, 4 and 5. The system checks the cache index to determine if the indices are stored in the cache space. For example, in our scenario, indices 4 and 5 are stored in DPU0's cache space, allowing us to read the MRAM once to obtain the partial sum $4 + 5$. This helps to reduce the number of memory accesses from two to one. Index 1 is not stored in the cache space, so we retrieve its embedding vector from the EMT space. Ultimately, each DPU returns a partial sum to the CPU (e.g., $4+5$ and 1). We combine and aggregate these partial sums using CPU to obtain final results.

In the above design, one parameter is the cache capacity. The cache space and EMT space share the 64MB MRAM size. The larger share for cache space, the higher reduction can be obtained on embedding lookup time. For example, using GoodReads dataset and varying the cache capacity to 40%, 70% and 100% of the required storage size of cache list, our cache-aware non-uniform partitioning is able to reduce the embedding lookup time by 17%, 22% and 26%, respectively, compared to that of without caching. The downside of storing large cache list is that it requires larger memory capacity. By default, we set the cache capacity to 100% of the required size.

4 EVALUATION**4.1 Experimental Setup**

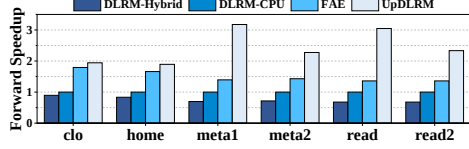
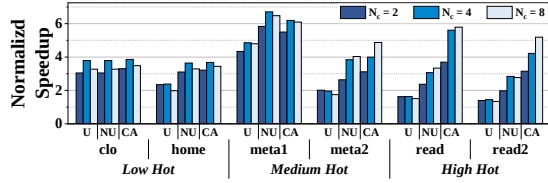
Workloads. We adopt Meta's deep learning recommendation model (DLRM) [13] with six real-world datasets¹, as shown in Table 1. The datasets can be categorized into three groups, namely low hot, medium hot and high hot, according to the average reduction frequency of the dataset. In our experiments, we duplicate each dataset to form eight EMTs and each embedding vector has 32 dimensions. To measure inference performance, we conduct a sampling of 12,800 inferences in each set of experiments. The batch size is set to 64.

Comparisons. We compare UpDLRM with three other open-source DLRM implementations, as detailed in Table 2. DLRM-CPU

¹https://cseweb.ucsd.edu/~jmcauley/datasets/amazon_v2
<https://mengtingwan.github.io/data/goodreads.html>

Table 2: Specifics of evaluated hardware architectures

Implementation	Architecture	CPU core	Memory
DLRM-CPU [13]	Intel Xeon(R) Silver 4110(2.10GHz)	32	128GB
DLRM-Hybrid [4]	Intel Xeon(R) Silver 4110(2.10GHz)	32	128GB
FAE [4]	NVIDIA GeForce GTX 1080 Ti	-	11GB
UpDLRM	Intel Xeon(R) Silver 4110(2.10GHz) UPMEM DPU(350MHz) × 256	32 -	128GB 16GB

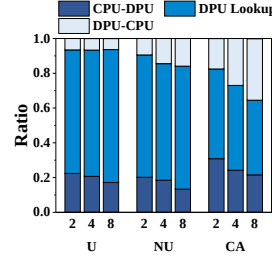
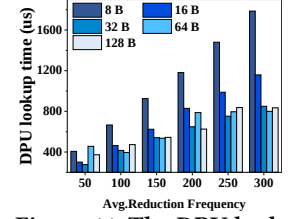
**Figure 8: Inference performance speedup of compared approaches over that of DLRM-CPU****Figure 9: Performance speedup of embedding layers obtained by three partitioning methods over DLRM-CPU**

is the CPU-only implementation that adopts CPU for both EMT storage and parallel computations. DLRM-Hybrid and FAE [4] are two implementations based on the CPU-GPU hybrid architecture. In the hybrid architecture, CPU is used to store EMTs and perform embedding lookups, while the GPU manages CTR computation. Embedding lookup results are communicated to the GPU via PCI-e. The FAE approach differs from DLRM-Hybrid in that it involves placing a subset of highly accessed item embedding vectors in a cache space, such as the GPU memory, to accelerate DLRM inference time. Our UpDLRM implementation is illustrated in Figure 4, where cache-aware EMT partitioning is used in the pre-process stage. We adopt two UPMEM modules, totalling 256 DPUs. Each DPU employs 14 tasklets.

4.2 Effectiveness of UpDLRM

Figure 8 illustrates the performance of the compared implementations on the six datasets, in terms of inference time speedup compared to that of DLRM-CPU. Overall, UpDLRM obtains the best speedup performance among all comparisons for all datasets. Specifically, UpDLRM accelerate the inference time by 1.9x-3.2x, 2.2x-4.6x and 1.1x-2.3x compared to DLRM-CPU, DLRM-Hybrid and FAE, respectively. Especially, it obtains higher speedup when the average reduction of the dataset is high. A larger average reduction represents a higher number of embedding lookup operations in a dataset. Thus, the results demonstrate that our DPU-based architecture is effective on improving the performance of embedding layers. Unsurprisingly, FAE outperforms DLRM-CPU and DLRM-Hybrid, demonstrating the effectiveness of its caching technique. DLRM-Hybrid performs the worst among all, mainly due to the fact that significant time is required by the CPU to execute embedding lookups, leading to GPUs waiting for the embedding results before proceeding with CTR computation.

To learn the improvements obtained by UpDLRM in details, we further perform comparative evaluation of the three proposed EMT

**Figure 10: Latency breakdown of embedding layers with three partitioning methods using Goodreads. N_c is fixed at 2, 4 or 8.****Figure 11: The DPU lookup time under varying average reduction frequencies and lookup data sizes (8B to 128B).**

partitioning methods, namely uniform (U), non-uniform (NU) and cache-aware (CA), using the six datasets. Figure 9 illustrates the performance speedup of the embedding layer obtained by the three methods compared to DLRM-CPU. The number of columns per partition (i.e., N_c) is fixed to 2, 4 or 8 for different partitioning methods. We have the following observations: 1) Under the same N_c value, CA partitioning obtains larger speedup compared to U and NU for High Hot datasets. This explains the good performance of UpDLRM under High Hot in Figure 8. 2) The three partitioning methods perform almost the same for the “clo” dataset. This is because the item access pattern in this dataset is quite balanced, and the cache rate is low. 3) There’s no universally good choice of N_c for different datasets. For example, $N_c = 4$ works the best for the first three datasets while $N_c = 8$ is optimal for the latter three. Recall that a large N_c value leads to higher DPU-CPU communication time but lower embedding lookup time and CPU-DPU communication time, as discussed in Section 3.1. UpDLRM can automatically achieve good balance between the two for different datasets.

4.3 Latency breakdown

To study the performance of each component of Figure 4, we further breakdown the latency of UpDLRM into three parts, namely CPU-DPU communication time (stage 1), DPU lookup time (stage 2) and DPU-CPU communication time (stage 3). Figure 10 shows the breakdown results when using uniform (U), non-uniform (NU) and cache-aware (CA) partitioning in pre-process stage of UpDLRM. N_c is fixed at 2, 4 and 8. We have the following observations.

First, comparing different partitioning methods, CA reduces the ratio of DPU lookup time in the embedding time from 71%-77% to 43%-52%, compared to the other partitioning methods. This means that the partial sum caching together with the cache-aware partitioning are effective on reducing the embedding lookup time in UpDLRM. With the ratio of embedding lookup time decreases to below 50%, it means the bottleneck effect of the embedding operations has been greatly mitigated.

Second, comparing the same partitioning method under different N_c values, the CPU-DPU communication time increases and DPU-CPU communication time decreases with the increase of N_c . For example, for CA, the CPU-DPU time ration reduces from 31%-21% and DPU-CPU time ratio increases from 17%-35% when N_c increases from 2 to 8. This is consistent with our analysis of UpDLRM performance in Section 3.1.

4.4 Sensitivity Study

In Figure 8, we have observed better performance of UpDLRM for datasets with higher average reduction. To better study the potential of UpDLRM, we create a set of synthetic datasets with balanced item access patterns and varying average reduction frequencies ranging from 50 to 300. We vary N_c from 2, 4, ..., to 32, thus resulting in varying data sizes of 8B, 16B, ..., to 128B. These sizes correspond to the amount of data loaded from MRAM per lookup. We set batch size to 64 samples. Figure 11 shows the DPU lookup time results under different configurations. We have the following observations.

First, under the same lookup data size, the DPU lookup time increases almost linearly with the increase of average reduction frequency. For example, when the lookup data size is 8B, the DPU lookup time increases from 406us to 1786us. When the data size gets larger, the increase of DPU lookup time with the increase of average reduction becomes rather stable. For example, when the lookup data size is 64B, the DPU lookup time increases from 456us to 787us when the average reduction increases from 50 to 200, and remains almost unchanged when the average reduction continues to increase to 300. This is because we employ 14 tasklets in one DPU, and the pipeline in high hot scenarios effectively masks the MRAM read latency, resulting in a comparable latency. The results demonstrate that, the embedding lookup performance of UpDLRM is quite scalable with the increase of average reduction frequency.

Second, given a fixed average reduction frequency, the DPU lookup time greatly reduces when the lookup data size increases from 8B to 32B. Recall our previous observation in Figure 3, the MRAM access latency remains almost unchanged when the access data size varies between 8B and 32B. With larger per lookup data size, less number of lookup operations is needed and thus lower lookup time can be obtained. When the access data size increases to larger than 32B, the per lookup latency also increases, thus hinders the reduction of DPU lookup time. Thus, in previous experiments, we only set N_c to 2, 4 and 8 to study the performance of UpDLRM.

5 RELATED WORK

PIMPR [19] utilizes ReRAM crossbar, using NOR gates within the ReRAM arrays to maintain the Computing-In-Memory capability, to enhance DLRM acceleration. However, its hardware structure differs from UPMEM PIM, the first commercial processing-in-memory chip. RecNMP [10] implements near-memory processing of the embedding vector around specialized DIMMs to expand the accelerator's memory space to tens of GBs. These studies are orthogonal to ours due to the different hardware architectures. EVStore [12] implements a three-degree cache method to store popular items. It utilizes mixed precision datatype to cache a larger number of embedding vectors and leverages the similarity between embedding vectors to further enhance the cache hit rate. In addition, GRACE [20] and SPACE [9] leverage hot items to reduce memory access, caching the hot embedding vector and partial sum in a cache space. However, they do not consider scenarios involving allocating the hot embedding vector in multiple cache spaces, such as caching in multiple DPUs, potentially leading to workload imbalance. UpDLRM introduces a cache allocation method across multiple cache spaces and addresses the workload imbalance issue arising after caching. Note that, although we adopt GRACE to generate cache

list in this paper, UpDLRM does not rely on GRACE and can work with any other caching technique.

6 CONCLUSION

This paper proposes *UpDLRM*, which utilizes real-world processing-in-memory (PIM) hardware, UPMEM DPU, to boost the memory bandwidth and reduce recommendation latency. The parallel nature of the DPU memory can provide high aggregated bandwidth for the large number of irregular memory accesses in embedding lookups, thus offering great potential to reduce the inference latency of DLRMs. To fully utilize the DPU memory bandwidth, we further studied the embedding table partitioning problem to achieve good workload-balance and efficient data caching. Evaluations using real-world datasets show that, UpDLRM achieves up to 4.6x speedup in terms of inference performance compared to both CPU-only and CPU-GPU hybrid counterparts. In the future, we plan to work on designing a DPU-GPU heterogeneous system to further optimize the inference time of DLRM systems.

ACKNOWLEDGEMENT

This work is supported by National Natural Science Foundation of China (62172282, 62293510, 62293513, 62272252, 62272253), Guangdong Natural Science Foundation 2022A151010122, Shenzhen Science and Technology Foundation RCYX20221008092908029, and a startup grant of HKBU.

REFERENCES

- [1] 2022. Meta Research. Embedding Lookup Synthetic Dataset. https://github.com/facebookresearch/dlrm_datasets.
- [2] 2023. Introducing the most advanced Processing In Memory product. <https://www.upmem.com/>.
- [3] 2023. UPMEM User Manual. <https://sdk.upmem.com/2023.1.0/index.html>.
- [4] Muhammad Adnan et al. 2021. Accelerating recommendation system training by leveraging popular choices. *arXiv preprint arXiv:2103.00686* (2021).
- [5] Paul Covington et al. 2016. Deep neural networks for youtube recommendations. In *RecSys'16*. 191–198.
- [6] Fabrice Devaux. 2019. The true processing in memory accelerator. In *HCS'19*. IEEE Computer Society, 1–24.
- [7] Udit Gupta et al. 2020. The architectural implications of facebook's dnn-based personalized recommendation. In *HPCA'20*. IEEE, 488–501.
- [8] Rishabh Jain et al. 2023. Optimizing CPU Performance for Recommendation Systems At-Scale. In *ISCA'23*. Article 77, 15 pages.
- [9] Hongju Kal et al. 2021. Space: locality-aware processing in heterogeneous memory for personalized recommendations. In *ISCA'21*. IEEE, 679–691.
- [10] Liu Ke et al. 2020. Recnmp: Accelerating personalized recommendation with near-memory processing. In *ISCA'20*. IEEE, 790–803.
- [11] Heewoo Kim et al. 2023. RecPIM: A PIM-Enabled DRAM-RRAM Hybrid Memory System For Recommendation Models. In *ISLPED'23*. 1–6.
- [12] Danian H Kurniawan et al. 2023. EVStore: Storage and Caching Capabilities for Scaling Embedding Tables in Deep Recommendation Systems. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 281–294.
- [13] Maxim Naumov et al. 2019. Deep learning recommendation model for personalization and recommendation systems. *arXiv preprint arXiv:1906.00091* (2019).
- [14] Maxim Naumov et al. 2020. Deep learning training in facebook data centers: Design of scale-up and scale-out systems. *arXiv preprint arXiv:2003.09518* (2020).
- [15] Jianmo Ni et al. 2019. Justifying recommendations using distantly-labeled reviews and fine-grained aspects. In *EMNLP-IJCNLP'19*. 188–197.
- [16] Jérémie Rappaz et al. 2021. Recommendation on live-streaming platforms: Dynamic availability and repeat consumption. In *RecSys'21*. 390–399.
- [17] Mengting Wan et al. 2018. Item recommendation on monotonic behavior chains. In *RecSys'18*. 86–94.
- [18] Mengting Wan et al. 2019. Fine-grained spoiler detection from large-scale review corpora. *arXiv preprint arXiv:1905.13416* (2019).
- [19] Tao Yang et al. 2023. PIMPR: PIM-based Personalized Recommendation with Heterogeneous Memory Hierarchy. In *DATE'23*. 1–6.
- [20] Haojie Ye et al. 2023. GRACE: A Scalable Graph-Based Approach to Accelerating Recommendation Model Inference. In *ASPLOS'23*. 282–301.

ChatCPU: An Agile CPU Design & Verification Platform with LLM

Xi Wang

xi.wang@seu.edu.cn
National ASIC Center, School of
Integrated Circuit, Southeast University
National Center of Technology Innovation
for Electronic Design Automation
Nanjing, Jiangsu, China

Gwok-Waa Wan*

Sam-Zaak Wong
Layton Zhang

{gwokwaa,samzaak,layton}@nctieda.com
National Center of Technology Innovation
for Electronic Design Automation
Nanjing, Jiangsu, China

Tianyang Liu

Qi Tian
Jianmin Ye

{liutianyang,qi_tian,jianmin_y}@seu.edu.cn
National ASIC Center, School of
Integrated Circuit, Southeast University
Nanjing, Jiangsu, China

Abstract

The increasing complexity of semiconductor designs necessitates agile hardware development methodologies to keep pace with rapid technological advancements. Following this trend, the Large Language Models (LLMs) emerge as a potential solution, providing new opportunities in hardware design automation. However, existing LLMs exhibit challenges in HDL design and verification, especially for complicated hardware systems. Addressing this need, we introduce ChatCPU, the first end-to-end agile hardware design and verification platform with LLM. ChatCPU streamlines the ASIC design and verification process, guiding it from initial specifications to the final RTL implementations with enhanced design agility. Incorporating the LLM fine-tuning and the processor description language design for CPU design automation, ChatCPU significantly enhances the hardware design capability using LLM. Utilizing ChatCPU, we developed a 6-stage in-order RISC-V CPU prototype, achieving successful tape-out using SkyWater 130nm MPW project with Efabless, which is currently the largest CPU design generated by LLM. Our results demonstrate a remarkable improvement in CPU design efficiency, accelerating the design iteration process by an average of 3.81X, and peaking at 12X and 9.33X in HDL implementations and verification stages, respectively. The ChatCPU also enhances the design capability of LLM by 2.63X as compared to base Llama2. These advancements in ChatCPU represent a significant milestone in LLM-driven ASIC design and optimization.

CCS Concepts: • Computing methodologies → Artificial intelligence; • Computer systems organization; • Hardware → Electronic design automation;

Keywords: ChatCPU, LLM, Agile Hardware Design, Verification

ACM Reference Format:

Xi Wang, Gwok-Waa Wan, Sam-Zaak Wong, Layton Zhang, Tianyang Liu, Qi Tian, and Jianmin Ye. 2024. ChatCPU: An Agile CPU Design & Verification Platform with LLM. In *61st ACM/IEEE Design Automation Conference (DAC '24)*, June 23–27, 2024, San Francisco, CA, USA. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3649329.3658493>

*Corresponding author: Gwok-Waa Wan

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

DAC '24, June 23–27, 2024, San Francisco, CA, USA

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0601-1/24/06

<https://doi.org/10.1145/3649329.3658493>

1 Introduction

In the era of big data, the surge in data-intensive workloads such as machine learning, graph analytics, blockchain, etc. shows the ever-increasing demand for enhanced computing capabilities. This need, alongside advancements in CMOS scaling and domain-specific architectures, has catalyzed an escalating expansion in the complexity of modern silicon designs. As a consequence, the process of constructing hardware prototypes for such systems from the ground up has become a formidable challenge, entailing substantial time and effort dedicated to implementations and verification.

Targeting this challenge, many efforts have been devoted to agile hardware designs to accelerate iterations. Recent approaches have leveraged High-Level Synthesis (HLS) [6] and modern Hardware Description Languages (HDLs) like Chisel and SpinalHDL [2], which can subsequently be compiled to RTL. However, these methods aim to enhance agility via additional layers of abstraction, resulting in difficulties of design verification and fine-grained control over the performance, power, and area (PPA) [12, 16].

Innovations in Machine Learning (ML) and Artificial Intelligence (AI) offer new avenues for agile hardware design [10]. Current ML applications target specific design stages such as the design space exploration, physical design, etc., yet their integration across the entire processor design cycle spanning from hardware specifications to HDL implementation is still emerging [15, 17, 18]. The processor design and verification process, particularly, remains labor-intensive, requiring significant expertise and extensive validation. The advent of Large Language Models (LLMs) [13] presents an opportunity in this domain. Recent efforts demonstrate the potential of LLMs in automating hardware implementations [3, 4]. However, the quality of LLM-generated hardware designs poses challenges in synthesis and functional verification, often leading to increased debugging and verification efforts. Moreover, existing LLM-based silicon generations remain tiny-scale, and LLMs exhibit limited capability in complex system designs like CPUs.

Therefore, we introduce **ChatCPU**, an agile CPU design and verification platform with fine-tuned LLM. ChatCPU features a hierarchical CPU design methodology and modularized HDL generation paradigm. We also designed a new Processor Description Language (PDL) dedicated to clear and unified CPU architecture descriptions, improving prompt management by eliminating ambiguity in design specifications. We further introduce an Agile Model Generator (AMG) for automated CPU reference model generation based on the CPU design description with PDL. Integrating the AMG and PDL, we propose a Module-interchangeable Verification Paradigm (MVP) to address the verification challenges of complicated hardware system designs using LLMs. Attributed to the enhanced intelligence and automation, ChatCPU substantially reduces design iteration time and effort, streamlining the lab-to-fab process of

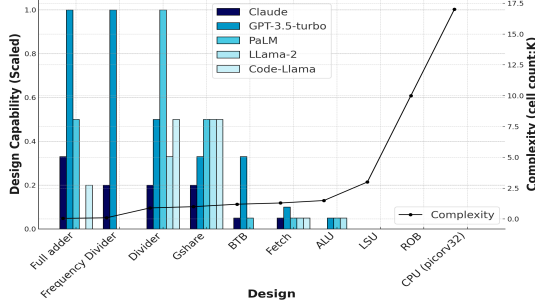


Figure 1. Hardware Design Capability with Different Complexities

hardware development. As a validation of the proposed framework, we designed *Saber6*, a 6-stage in-order RISC-V CPU with ChatCPU and implemented the silicon prototype with OpenEDA flow [14] and SkyWater 130nm MPW project. As a case study, we utilized ChatCPU to design and implement a RISC-V CPU prototype with the OpenEDA flow [14] and the MPW project, demonstrating the potential of ChatCPU to revolutionize CPU design and verification.

This research makes six key contributions.

- We extensively analyze the RTL design capability of existing LLMs and summarize the challenges of LLM-based design.
- We introduce ChatCPU, the first CPU design and verification platform with LLM, featuring a high degree of extensibility and scalability. ChatCPU is capable of autonomously driving end-to-end ASIC design.
- We construct the CPU design dataset and fine-tune the Llama 2 to enhance the capability of RTL generation.
- We design a processor design language and an agile model generator to automate the generations of CPU designs and reference models in a homologous manner.
- We propose a module-interchangeable verification paradigm to drive complete CPU verification flow upon each CPU module generated by LLM, based on the co-simulation support between modularized RTL design and CPU reference model.
- We design and verify a RISC-V CPU from scratch with ChatCPU, and fabricate the silicon prototype with OpenEDA and SkyWater 130 OpenPDK [7] in the Efabless MPW project.

The rest of this paper is organized as follows. Section 2 presents the motivation of ChatCPU and related work. Section 3 describes the detailed ChatCPU design and workflow. Section 4 presents the evaluations of the ChatCPU, and Section 5 concludes the paper.

2 Background

2.1 LLM in Agile Hardware Design

Recent advancements in LLMs such as BERT, GPT-2, and GPT-3 have showcased significant capabilities in Natural Language Processing (NLP) and text generation. These models utilize contextual information effectively, offering the potential for generating intricate hardware designs from high-level natural language inputs [9]. Initiatives like ChipGPT [4] and ChipChat [3] take advantage of LLMs for hardware design. However, ChipGPT focuses more on optimizing prompts rather than tackling the broader challenges of agile hardware design [19]. Conversely, ChipChat, while advancing LLMs in chip design, has only demonstrated results on a small scale, approximately 1,000 logic gates, which pales in comparison

to modern processors that incorporate billions of gates. This disparity raises questions about the scalability and practicality of these methods for more complex designs.

In addition to design generation, verification remains a crucial aspect of AI-driven hardware design paradigms. For example, a recent study explored AI-driven CPU design based solely on external input-output observations with extensive I/O operations for circuit matching, rather than formal program code. This approach, while effective within its scope, results in a 'black-box' design process, leading to potential redundancies and challenges in design verification [5]. Current LLMs, despite their advancements, still produce hardware designs that are prone to errors and may not pass synthesis and functional checks, thereby inducing overhead in verification. Given the high costs associated with design verification in modern CPUs, it is imperative that agile hardware design not only automates HDL generation but also streamlines the verification.

2.2 LLM Hardware Design Analysis

To assess the efficacy of LLMs in hardware design generation, we conducted an evaluation using Claude, GPT3.5, PaLM, Llama2, and Code-LLama. Our test involved generating ten different open-source hardware designs written in Verilog, with complexity ranging from a simple adder to a sophisticated CPU. Each LLM received identical design specifications, and we monitored their results of synthesis and functionality verification. If errors were detected, the LLMs were tasked with debugging and regenerating the code. We quantified the design capability (C) of each LLM using the metric,

$$C = \frac{1}{N} \quad (1)$$

where N represents the number of iterations needed by the LLM to pass both synthesis and verification.

Following this metric, a higher C value indicates superior design capability. We allowed a maximum of 20 iterations for each design. If an LLM failed to produce a viable design within these iterations, its design capability for that particular design was recorded as zero. As depicted in Figure 1, GPT3.5 demonstrated the highest design capability among the evaluated LLMs. However, we observed a consistent decline in design capability across all LLMs as design complexity increased, particularly evident when transitioning from simpler designs to those comprising over 17,000 cells, such as the PicoRV32 CPU. Notably, none of the LLMs succeeded in generating functional Load Store Units (LSU), Reorder Buffers (ROB), or the PicoRV32 CPU designs, underscoring the challenges LLMs face in handling complex system designs.

2.3 Problem Formulation

Building upon the results and analysis from Section 2.2, we identify and define three primary challenges ($C1$ – $C3$) in LLM-based hardware design:

- **C1. Limited HDL Generation Capability.** The hardware designs generated by LLMs frequently fail to meet synthesis and functional verification standards, indicating a gap in the quality of generated code.
- **C2. Limitations in Complex System Design:** While LLMs show promising capability in generating designs for simpler

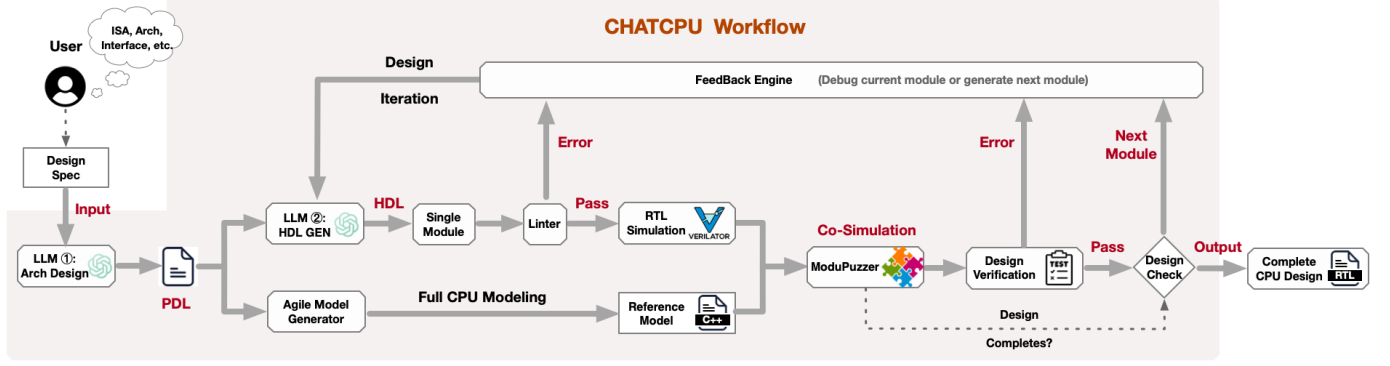


Figure 2. ChatCPU Architecture and Workflow

modules, their effectiveness diminishes with increasing complexity. This limitation becomes particularly evident in intricate hardware systems where the success rate of effective design generation by LLMs significantly decreases.

- **C3. Design Verification Challenges:** LLMs struggle to adequately address bugs within complex, interconnected modules. The limitations in token and chat history further constrain LLMs from comprehensively covering both the complete codebase and the intricate logic required in large-scale HDL designs.

3 Design & Philosophy

3.1 Solutions & Strategies

In response to the hardware design challenges (C1–C3) of LLMs identified in Section 2.3, we propose the ChatCPU design, incorporating the following key solutions:

- **S1. LLM Fine-Tuning:** To improve the quality of HDL code generation, we refine LLMs using high-quality training datasets derived from existing open-source hardware designs. This involves data collection, cleaning, and labeling to enable more accurate and effective HDL generation.
- **S2. Hierarchical System Design with Divide & Conquer Approach:** Addressing the complexity of CPU system design, we decompose it into smaller, manageable hardware modules. This hierarchical structuring allows for the distribution of complex design tasks across specialized LLMs, each assigned specific roles, thereby enhancing design efficiency and manageability.
- **S3. Processor Description Language (PDL)-Based Design Generation:** We introduce a novel PDL to standardize CPU design specifications. This unified language serves as the foundation for automated design and reference model generation, facilitating both the design and verification.
- **S4. Module-Interchangeable Verification Paradigm (MVP):** To tackle the verification challenges of intricate designs produced by LLMs, we implement the MVP methodology. This approach integrates each generated module into a pre-verified reference model, enabling a comprehensive design verification flow.

3.2 HDL Selection

HLS offers the convenience of abstracting complex RTL design details through high-level languages like C/C++, but it limits control over hardware implementations and PPA tradeoffs. Besides, tracing issues from HLS to the original code is challenging, complicating debugging and verification. Similarly, while Chisel and SpinalHDL have their advantages, they present limitations in design verification, such as variable renaming and inadequate assertion support, respectively. Therefore, in this work, we select traditional HDLs (Verilog/SystemVerilog) for ChatCPU, leveraging their robustness and transparency in design and verification processes.

3.3 LLM Fine-Tuning

To enhance LLM capabilities in RTL code generation, we utilized Google Big Query to collect open-source RTL designs from GitHub, forming our primary dataset. This dataset was then processed with GPT4 to generate appropriate commands for each RTL segment, followed by deduplication and removal of irrelevant data. We trained these question-answer pairs using the Llama2-70B model, aiming to align RTL code with its functional descriptions. This approach enables the LLM to comprehend not only the RTL syntax but also the underlying functional context. Furthermore, we incorporated our PDL design specifications and detailed examples into the training set. This addition aims to boost the LLM proficiency in generating HDL and PDL content based on design specifications. During the fine-tuning phase, we employed the Low-Rank Adaptation (LoRA) technique, which targets a limited number of parameters, assuming a low “intrinsic rank” of weight changes during model tuning. This method optimizes the low-rank decomposition of weight matrices in self-attention modules, thereby streamlining the optimization process from higher to lower dimensions.

3.4 ChatCPU Workflow

As shown in Figure 2, ChatCPU employs a hierarchical design and modular generation flow with two fine-tuned LLMs featuring distinct roles. The first LLM (LLM ①) is tasked with translating design specifications into high-level architecture designs using our Processor Description Language (PDL), detailing aspects such as module lists, pipeline stages, and pin-to-pin connections. This PDL file is then fed into the Agile Model Generator (AMG) to automatically produce a cycle-accurate CPU reference model for design verification. Meanwhile, the second LLM (LLM ②) processes the PDL for

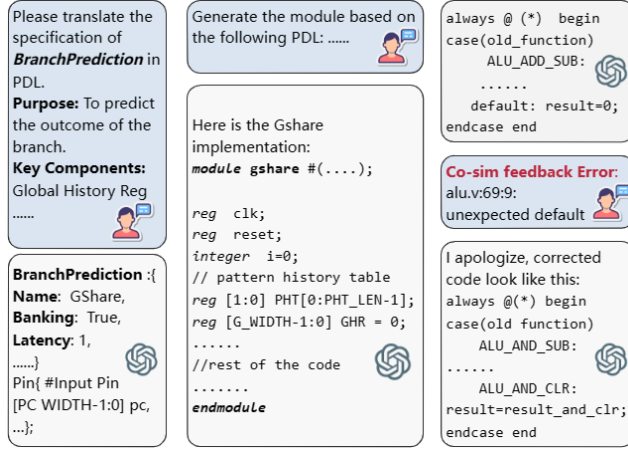


Figure 3. Prompt Examples of ChatCPU. From left to right are examples of PDL generation, HDL generation, and debugging.

each module, iteratively generating detailed HDL implementations. After a module passes linter checks, it is then verilated to a corresponding RTL model for co-simulation with the CPU reference model. This process, part of the MVP, allows for cross-validation of functionality. If the verification is successful, the design checker prompts LLM ① via a feedback engine to proceed with the next module generation. Any errors identified during code linting or design verification are reported back to LLM ② for module redesign.

Figure 3 provides examples of prompt interactions in ChatCPU. Part (a) of the figure illustrates how LLM ① receives a specification input and produces a corresponding PDL output for a Branch Prediction Unit (BPU) design. Part (b) demonstrates the design generation based on the provided PDL in LLM ②. Lastly, part (c) of the figure shows a scenario where issues are encountered during the verification process, highlighting the interactive and iterative nature of the design and verification cycle in ChatCPU.

3.5 Processor Description Language

Within ChatCPU, the processor description language (PDL) is innovatively crafted to encapsulate the timing, functionality, pipeline architecture, and pin-to-pin interfaces of each CPU module. PDL effectively translates CPU design specifications, originally in natural language, into a unified, machine-readable format. This comprehensive representation in the PDL file enables it to act as a foundational input for both the generation of HDL implementations and the construction of the CPU reference model. Figure 3(a) provides an illustrative example of PDL usage in CPU design specification.

3.6 Agile Model Generator

As depicted in Figure 4, the agile model generator (AMG) within ChatCPU initiates its process by interpreting PDL input to define the CPU timing behavior, functional characteristics, and pipeline structure. AMG facilitates interactions between modules through a well-defined timing interface, ensuring precise representation of timing relations across different modules.

The core of AMG operation is a decoupled timing and functional modeling mechanism, utilizing an event-based simulation approach to enhance both configurability and scalability. In this system, an

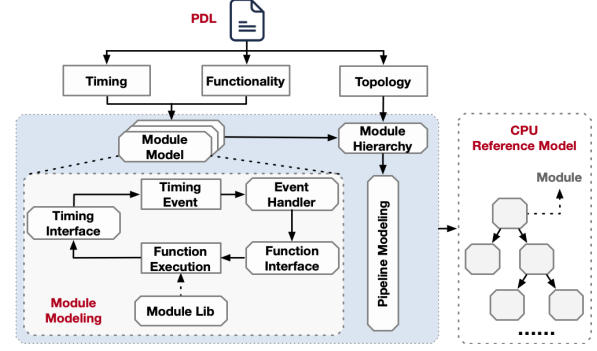


Figure 4. Agile Model Generator Workflow

event handler correlates timing events with their corresponding functional simulations via a functional interface. AMG module library houses functional implementations for simulating the behavior of classical modules in modern CPU designs. Upon completing the functional simulation, the timing interface is signaled to schedule subsequent events. The final stage of AMG's workflow involves connecting the established modules in a pipeline, culminating in the generation of a complete reference model for the CPU.

3.7 MVP Verification Flow

Verification plays a pivotal role in ASIC design, particularly for designs generated by LLMs. Addressing the verification challenge (C3) outlined in Section 2.3, we introduce the Module-interchangeable Verification Paradigm (MVP), specifically tailored for LLM-based CPU design. An example of this flow is shown in Figure 5.

The CPU reference model serves as the cornerstone for design verification. The reference model generated by ChatCPU offers a modularized golden model for both module-level and comprehensive CPU verification. We first ventilate each generated module to a respective RTL model for module-level simulation. The verification process starts with each LLM-generated module being translated into an RTL model using Verilator, a tool that converts RTL designs into C++ for simulation. Module-level verification is then conducted through unit tests, with results cross-validated against the equivalent module in the CPU reference model. Modules successfully passing unit tests, akin to fitting pieces in a jigsaw puzzle, are then integrated into the reference model. This integration is facilitated by co-simulation techniques such as Verilog Procedural Interface (VPI), Direct Programming Interface (DPI), or Transaction-Level Modeling (TLM), ensuring seamless module interchangeability.

Given the unified design language provided by PDL, the interchanged modules align perfectly with the reference model. The subsequent co-simulation of the verified RTL model with the reference model allows for comprehensive functionality evaluations. This includes ISA tests, torture tests, benchmarks, and real-world application simulations. As the CPU reference model is pre-verified for functional correctness, any discrepancies identified during co-simulation can be accurately attributed to the newly integrated RTL module. This targeted approach significantly streamlines the debugging process, overcoming the limitations of LLM-generated designs related to token and chat history constraints. The MVP approach thus offers a modular yet holistic verification strategy,

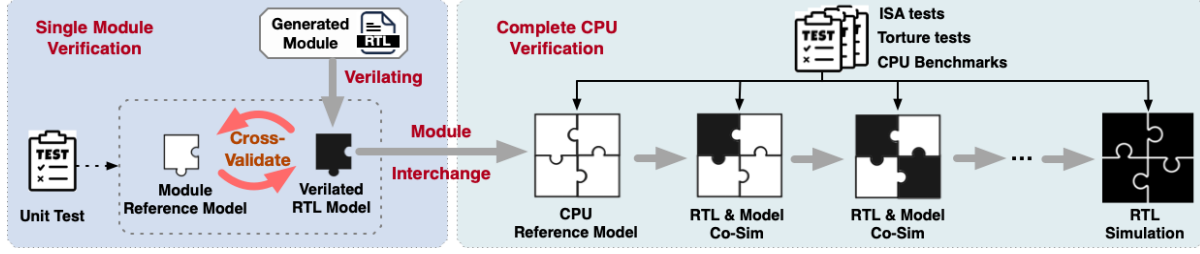


Figure 5. Example of Module-interchangeable Verification Flow: From Single Module to Complete CPU

ideally suited for the LLM-driven, modularized CPU design process, effectively addressing the complexity of verifying intricate systems.

4 Evaluations

Our experimental environment is delineated in Table 1. To validate our methodologies, we designed a 6-stage in-order RISC-V CPU: *Saber6*, with ChatCPU from scratch and completed the tapeout with SkyWater 130nm MPW project, which is so far the largest and most complicated CPU design (80K gates) generated by LLMs. The layout and detailed PPA information can be found in Figure 12.

To validate our methodologies, we designed a 6-stage in-order RISC-V CPU, named *Saber6*, using ChatCPU. This CPU, comprising approximately 80,000 gates, represents the largest and most complex CPU design generated by LLMs to date. The tapeout was completed with the SkyWater 130nm MPW project, and the layout and detailed PPA information are illustrated in Figure 12.

Table 1. Experimental Environment Specifications

Server	NVIDIA Server@Teax,US
CPU	AMD EPYC 7V13 64-Core Processor
GPU	4xA100(80GB,PCIe4.0)
Memory	256GB
CUDA	12.0
Disk	1024GB
OS	Ubuntu 22.04

4.1 Design Capability Analysis

To quantify the enhancement in design capability achieved through LLM fine-tuning, we utilized the RTLLM benchmark [11], comparing the performance of the default Llama2 and the fine-tuned LLM ② in ChatCPU. The design capability metric, as defined in Equation 1, is represented in Figure 6. Results indicate a significant improvement, with ChatCPU achieving an average design capability 2.63X higher than the base Llama2. Notably, ChatCPU was capable of generating all tested hardware modules, while the basic Llama2 was limited to smaller-scale designs.

Further, to assess the versatility of our fine-tuned LLM, we tasked LLM ② with optimizing various RTL designs, as shown in Figure 7. These designs, sourced from the OpenLane project [1], included diverse hardware modules such as GCD, USB interface, Zipdiv, Picorv32, and a co-processor for encryption. After collecting baseline power and area statistics using the OpenLane EDA flow, we directed the fine-tuned LLM to optimize these designs without altering their functionality. The optimization yielded significant reductions in power and area, averaging 16.70% and 13.69%, respectively, with peaks reaching up to 36.36% and 27.72%.

4.2 Design Agility Analysis

To evaluate the acceleration in design processes offered by ChatCPU, we conducted an experiment involving 12 graduate students, divided into four groups. One group utilized ChatCPU (G1), while the others did not use any LLMs (G2-G4). The time cost for each group to complete identical CPU design tasks is visualized in Figure 8. G1 with access to ChatCPU demonstrated markedly faster completion times, especially in code correctness and verification phases. On average, ChatCPU users experienced a 3.81X time saving, with notable efficiency gains of up to 12X in RTL coding and 9.33X in design verification. These results validate the advancements our methodology offers in expediting code generation, improving verification success rates, and enhancing overall development efficiency.

4.3 Design Verification and Iterations

Our analysis also focused on the impact of the MVP on design capability. We assessed the capability of ChatCPU in generating various CPU modules in *Saber6*, comparing scenarios with MVP enabled and disabled (using only unit tests). The results, shown in Figure 9, reveal that while MVP leads to more iterations in simpler hardware designs (due to its thorough CPU verification flow), it significantly reduces the number of iterations required for complex designs. This reduction is particularly notable in tightly coupled modules of CPU frontend, backend, and full CPU designs, where the MVP comprehensive verification process is critical. Conversely, without MVP, even a fine-tuned LLM struggles with the challenges of cross-module verification in complex systems.

Additionally, we evaluated the impact of employing PDL in ChatCPU. Comparing the design capability with and without PDL usage (Figure 10) demonstrates that using PDL enhances design capability by an average of 2.06X over human-written specifications.

4.4 Design Complexity Analysis

Finally, to further assess design complexity, we compared the cell count of *Saber6* with other LLM-generated hardware designs, including entries from the Efabless AI Design Contest [8]. As depicted in Figure 11, *Saber6* exhibits a significantly higher level of complexity than the other designs in terms of the cell count. This comparison underscores the effectiveness of ChatCPU in facilitating the generation of complex hardware systems, highlighting its potential to revolutionize agile hardware design methodologies.

5 Conclusion

In this work, we first investigated the LLM-based hardware designs and summarized the associated challenges. Targeting these challenges, we proposed the ChatCPU platform to enhance the agile

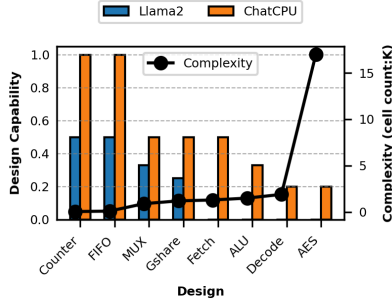


Figure 6. Impacts of LLM Fine-Tuning

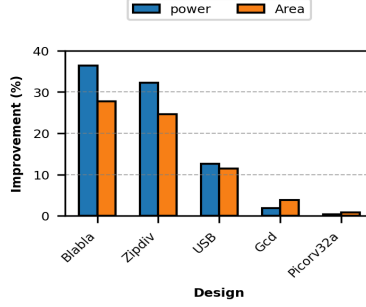


Figure 7. RTL Design Optimization

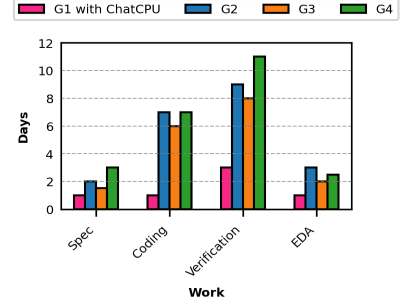


Figure 8. Time Cost Distribution

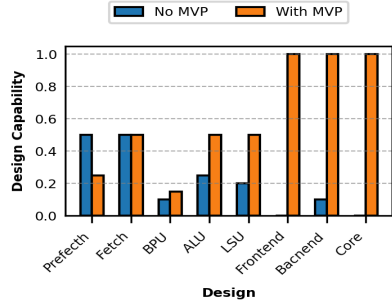


Figure 9. Verification Analysis

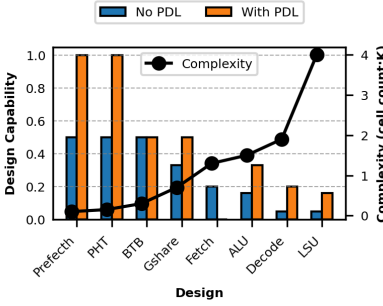


Figure 10. Impacts of PDL

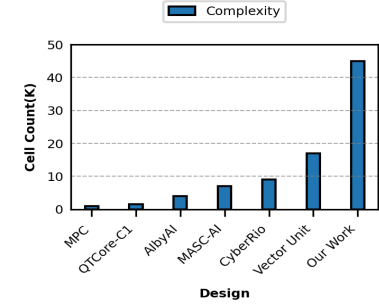
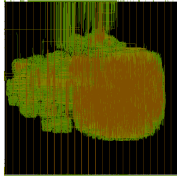


Figure 11. Complexity Comparisons



Feature	Parameters
PDK	Skywater 130nm
Gate Count	80,000
Pipeline	6
Frequency	100MHz
Total Power	19.2mW
Area	0.18mm ²

Figure 12. Silicon Layout and Information.

CPU design and verification using LLM. We proposed a hierarchical CPU design methodology with fine-tuned LLM, featuring distinct roles in the modularized CPU generation paradigm. We also introduced a Module-interchangeable Verification Paradigm (MVP) to effectively verify the hardware designs generated by LLMs. ChatCPU also integrates the OpenEDA flow to enable automated PPA optimizations, achieving an impressive boost in RTL-GDS efficiency by an average of 3.81X. Finally, we designed the largest RISC-V CPU generated by LLMs, with a scale of 80K gates using ChatCPU as a silicon prototype to validate our methodologies. exploit the transformative potential of LLMs in complex CPU designs.

References

- [1] 2023. OpenLane Project. <https://github.com/The-OpenROAD-Project/OpenLane>.
- [2] Jonathan Bachrach, Huy Vo, Brian Richards, Yunsup Lee, Andrew Waterman, Rimas Avizienis, John Wawrzyniak, and Krste Asanovic. 2012. Chisel: constructing hardware in a scala embedded language. In *DAC*.
- [3] Jason Blocklove, Siddharth Garg, Ramesh Karri, and Hammond Pearce. 2023. Chip-Chat: Challenges and Opportunities in Conversational Hardware Design. *arXiv preprint arXiv:2305.13243* (2023).
- [4] Kaiyan Chang, Ying Wang, Haimeng Ren, Mengdi Wang, Shengwen Liang, Yinhe Han, Huawei Li, and Xiaowei Li. 2023. ChipGPT: How far are we from natural language hardware design. *arXiv preprint arXiv:2305.14019* (2023).
- [5] Shuyao Cheng, Pengwei Jin, Qi Guo, Zidong Du, Rui Zhang, Yunhao Tian, Hu, et al. 2023. Pushing the Limits of Machine Design: Automated CPU Design with AI. *arXiv preprint arXiv:2306.12456* (2023).
- [6] Philippe Coussy and Adam Morawiec. 2010. *High-level synthesis*. Vol. 1. Springer.
- [7] R Timothy Edwards. 2020. Google/SkyWater and the Promise of the Open PDK. In *Workshop on Open-Source EDA Technology*.
- [8] efabless. 2023. Efabless AI Generated Open-Source Silicon Design Challenge. <https://efabless.com/ai-generated-design-contest>.
- [9] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, et al. 2020. Codebert: A pre-trained model for programming and natural languages. *arXiv preprint arXiv:2002.08155* (2020).
- [10] Guyue Huang, Jingbo Hu, Yifan He, Jialong Liu, Mingyuan Ma, Zhaoyang Shen, Juejian Wu, Yuanfan Xu, et al. 2021. Machine learning for electronic design automation: A survey. *ACM TODAES* (2021).
- [11] Yao Lu, Shang Liu, Qijun Zhang, and Zhiyao Xie. 2023. RTLLM: An Open-Source Benchmark for Design RTL Generation with Large Language Model. (2023).
- [12] Zohar Manna and Richard Waldinger. 1979. Synthesis: Dreams→Programs. *IEEE Transactions on Software Engineering* 4 (1979), 294–328.
- [13] Katharine Sanderson. 2023. GPT-4 is here: what scientists think. *Nature* 615, 7954 (2023), 773.
- [14] Mohamed Shalan and Tim Edwards. 2020. Building OpenLANE: a 130nm openroad-based tapeout-proven flow. In *ICCAD*.
- [15] Nan Wu, Yuan Xie, and Cong Hao. 2022. IronMan-Pro: Multiobjective Design Space Exploration in HLS via Reinforcement Learning and Graph Neural Network-Based Modeling. *IEEE TCAD* (2022).
- [16] Yanan Xu, Zihao Yu, Dan Tang, Guokai Chen, Lu Chen, Lingrui Gou, Yue Jin, Qianruo Li, Xin Li, Zuojun Li, et al. 2022. Towards developing high performance RISC-V processors using agile methodology. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, Chicago, IL, USA, 1178–1199.
- [17] Xiaoling Yi, Jialin Lu, Xiankui Xiong, Dong Xu, Li Shang, and Fan Yang. 2023. Graph representation learning for microarchitecture design space exploration. In *2023 60th ACM/IEEE Design Automation Conference (DAC)*. IEEE, IEEE, San Francisco, CA, USA, 1–6.
- [18] Jianwang Zhai and Yici Cai. 2023. Microarchitecture Design Space Exploration via Pareto-Driven Active Learning. *IEEE TVLSI* (2023).
- [19] Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitit, Harris Chan, and Jimmy Ba. 2022. Large language models are human-level prompt engineers. *arXiv preprint arXiv:2211.01910* (2022).